
Artificial Intelligence/ Advanced Artificial Intelligence
CSC-446-01 / CSC-546-01 / CSC-546-W1

Assignment 1- Winter 2026

Deadline: 01/31/2026 @ 23:59

[Total Mark for this Assessment is 100]

Student Details:

Name: Bradley Moore CRN: CSC 546	ID: 35737785
---	---------------------

Submission Instructions:

- This assignment must be submitted on Moodle **pdf format only**).
- The page number should appear on each page.
- All the text and diagrams must be electronically produced.
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented; marks may be reduced for poor presentation. This includes filling your information on the cover page.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- The assignment proposal should contain the following sections:

Question one - Ask a Language a Question

Purpose:

This assignment helps you understand why programming languages make different design choices. You will compare how three languages answer the same programming question, using an LLM only as a helper, not as the main work.

What You Will Do:

1. Choose ONE Question

Pick one question from the list below:

- How does this language handle errors?
- **How do functions work in this language?**
- How does this language manage data and state?
- How does this language prevent or allow bugs?
- How does this language support parallelism or concurrency?

2. Choose THREE Programming Languages

Select three different languages. You do not need to master the languages. Short examples are enough. (Java, Python, Perl)

To decide which 3 languages I wanted to investigate I have Copilot a simple prompt to return the top three programming languages used today.

Prompt: "What are the top 3 programming languages used today?"

Response: Based on the most recent large-scale developer data, three languages consistently dominate real-world usage across surveys and industry rankings:

Top 3 Most Used Programming Languages Today

1. JavaScript

- Continues to be the most widely used language among developers.
- Used by **62.3%** of developers in the past year according to the 2024 Stack Overflow Developer Survey.

2. Python

- Appears at or near the top of multiple popularity rankings, including IEEE Spectrum's 2024 list, which highlights Python as a leading language across domains such as AI, data science, and general software development.

3. SQL

- Also ranked highly in IEEE Spectrum's 2024 analysis, reflecting its essential role in data engineering, analytics, and backend systems.

These three languages show up repeatedly across independent surveys and rankings, making them the most consistently used and in-demand languages today.

If you want, I can break down what each language is best suited for or map them to different career paths

Based on this response from the LLM I decided that Javascript, Python, and SQL would be satisfactory to investigate deeper.

3. Use an LLM

You must:

3.1. Ask an LLM to explain how each language answers your question

Prompt:

For each of these 3 programming Languages I have been tasked to investigate: "How do Functions work in these Languages?" Please provide information regarding the questions and small code examples to highlight the explanation.

Explanation Response:

Below is a clean, comparative breakdown of **how functions work in JavaScript, Python, and SQL**, with **small, illustrative code examples**. The goal is to give you a conceptual model for each language's function system—how they're defined, how they're called, and how they behave internally.

■ 1. JavaScript — Functions as First-Class Citizens

JavaScript treats functions as **values**. They can be:

- Stored in variables

- Passed as arguments
- Returned from other functions
- Attached to objects

This makes JavaScript extremely flexible and heavily functional in style.

How Functions Work in JavaScript

- Declared using function keyword, arrow syntax, or as object methods.
- Functions form **closures**, meaning they remember variables from their creation scope.
- They can be invoked with any number of arguments (extra arguments are ignored; missing ones become undefined).

2. Python — Functions as Objects with Strong Structure

Python functions are also **first-class objects**, but the language emphasizes readability and explicitness.

How Functions Work in Python

- Defined using `def`.
- Support **default arguments**, **keyword arguments**, and **variable-length arguments**.
- Functions capture outer-scope variables via **closures**.
- Everything is an object, including functions.

3. SQL — Functions as Database Operations

SQL is not a general-purpose language, so “functions” behave differently. They fall into two categories:

A. Built-in SQL Functions

These operate on rows or sets of rows:

- `SUM()`, `AVG()`, `COUNT()`
- `UPPER()`, `LOWER()`
- `NOW()`, `DATEADD()`

They are **not user-defined code blocks** but predefined operations executed by the database engine.

🔗 How SQL Functions Work Conceptually

- They run **inside the database engine**, not in an application runtime.
- They cannot modify database state (in most SQL dialects).
- They are optimized for set-based operations, not procedural logic.

3.2. Request small code examples

Code Example Response:

Javascript:

💎 Example: Basic Function

```
function add(a, b) { return a + b; } console.log(add(2, 3)); // 5
```

💎 Example: Arrow Function

```
const multiply = (x, y) => x * y;
```

💎 Example: Closure

```
function makeCounter() { let count = 0; return function () { count++; return count; }; } const counter = makeCounter(); counter(); // 1 counter(); // 2
```

Python:

💎 Example: Basic Function

```
def add(a, b): return a + b print(add(2, 3)) # 5
```

💎 Example: Default & Keyword Arguments

```
def greet(name="World"): return f"Hello, {name}!" greet() # "Hello, World!"  
greet(name="Bradley") # "Hello, Bradley!"
```

💎 Example: Closure

```
def make_counter(): count = 0
def inner(): nonlocal count
count += 1
return count
return inner
counter = make_counter()
counter() # 1
counter() # 2
```

SQL:

💎 Example: Built-in Function

```
SELECT UPPER(first_name), LENGTH(last_name) FROM employees;
```

B. User-Defined Functions (UDFs)

These vary by database (SQL Server, PostgreSQL, MySQL).

They allow you to encapsulate logic and return a value.

💎 Example: SQL Server Scalar Function

```
CREATE FUNCTION AddNumbers (@a INT, @b INT) RETURNS INT AS BEGIN
RETURN @a + @b; END;
```

💎 Using the Function

```
SELECT dbo.AddNumbers(2, 3); -- 5
```

🧠 Summary Table

🧠	Summary Table		
Lang uage	How Functions Work	Key Features	Example Use
JavaS cript	Functions are values; closures are core	Arrow functions, callbacks, async functions	Web apps, event handlers
Pyth on	Functions are objects; explicit structure	Default args, decorators, closures	Data science, automation
SQL	Functions operate on data inside the DB	Built-in functions + UDFs	Aggregation, transformation

If you want, I can also create a **side-by-side comparison of syntax**, or expand into **advanced function features** like decorators (Python), higher-order functions (JS), or stored procedures (SQL).

3.3. Correct, clarify, or disagree with at least one LLM explanation among three languages (Discuss your own point of view about the proposed definition and examples provided by the LLM). Using LLM output without explanation or critique will reduce your grade.

I have used Copilot heavily for coding projects and conceptual understanding of a range of topics. Unfortunately, this leads to it's initial answer being heavily technical almost assuming as if we touched on this topic months ago that I should have some idea of these highly technical concepts.

For instance, when it gives the conceptual representation of how sequel works it explains it in relation to SQL itself.

How SQL Functions Work Conceptually

- They run **inside the database engine**, not in an application runtime.
- They cannot modify database state (in most SQL dialects).
- They are optimized for set-based operations, not procedural logic.

It may have been better served rooted in a concept more universally understandable. Although I have integrated SQL databases into many of my projects, they have been solely as tools to help the project, mainly interacted with libraries in the main language. In no way do I understand SQL Jargon like “database state” and “**inside the database engine**, not in an application runtime”.

Generally when this happens I will ask it for explanations outside of the technical context it returns:

Prompt:

“The Conceptual port of the SQL response was too rooted in technical jargon. Could you break it down in a different way”

Reponse:

“Absolutely — let’s strip away the jargon and explain **SQL functions** in a way that feels more intuitive and grounded in how SQL is actually used.

How SQL Functions Work — A Simpler, More Human Explanation

Think of SQL as a language that talks **directly to a database**.

Because of that, SQL functions behave very differently from Python or JavaScript.

Here’s the cleanest way to understand them:

★ SQL Functions Are “Little Helpers” Inside the Database

In SQL, a function is basically a **small tool** the database uses to transform or calculate something.

You can think of them like:

- A calculator built into the database
- A text formatter
- A date helper
- A custom rule you define

They don’t run in your application.

They run **inside the database engine**, right next to the data.

1 Built-In SQL Functions

These are the database’s **pre-installed tools**.

✓ They answer questions like:

- “What’s the total?” → SUM()
- “How many rows?” → COUNT()
- “Make this uppercase.” → UPPER()
- “How long is this text?” → LEN() or LENGTH()

Example

```
SELECT UPPER(first_name), LEN(last_name), salary * 1.05 AS new_salary FROM employees;
```

This is SQL's version of "do some quick math or formatting for me."

2 User-Defined Functions (UDFs)

These are **custom tools you build** when the built-in ones aren't enough.

A UDF is like telling the database:

"Hey, here's a small rule or formula I want you to remember.

Whenever I call it, run that rule and give me the result."

✓ What they're good for:

- Reusing a calculation
- Cleaning up repeated logic
- Making queries easier to read

🔗 Example (SQL Server)

```
CREATE FUNCTION AddNumbers (@a INT, @b INT) RETURNS INT AS BEGIN RETURN @a + @b; END;
```

Using it:

```
SELECT dbo.AddNumbers(2, 3);
```

This is SQL's version of "store this formula so I don't have to rewrite it."

🧠 The Real Mental Model (No Jargon)

Here's the simplest way to think about SQL functions:

SQL functions are **helpers**, not "programs."

They're meant to make queries cleaner, not to run complex workflows.

🔗 Why SQL Functions Feel Different from Python/JS

- Python and JavaScript functions run in your program.
- SQL functions run *inside the database*.
- SQL is designed for **asking questions about data**, not building software logic.

- So SQL functions are more like **formulas** than **code blocks**.

If you want, I can also rewrite this as:

- a teaching analogy
- a visual diagram
- a side-by-side comparison with Python/JS functions

Just tell me which direction you want to explore."

4. Explain the Differences

In your own words, explain how each language answers the question selected in step 1?

From what I understand it seems like Javascript is a much more literal, explicit, and rigid language when it comes to functions. The syntax looks very technical, leading it to be, at first glance, possibly harder to learn but generally there wouldn't be a question as to what the function is doing. It spells out intention clearly both declaring the types of variables entering the function and declaring the variable returning.

Python looks much more dynamic than Javascript. The function looks almost more like natural language. While Javascript requires variables to be declared and returned as the correct type, it looks like python allows variables to become whatever the functions return to them. Arguments can be passed to the function in any form and returned in any form the function wants. While this can be very powerful and dynamic, it can also lead to errors being harder to trace as it is very possible to believe you are doing something you are not within a function and returning or passing values in forms they are not.

SQL is sort of in a class of it's own. It is a database creator and transformer that is heavily used in Data Analysis. It's Syntax to me seems very abstract and is probably developed around Jargon relative to Database and Data Analysis. These functions are more of helper functions, they are meant you run efficient database queries, calculations, and transformations for your program that you are writing. These functions act only to query, append, and transform the data itself, it does not have things to manipulate the data. This way no one can inadvertently taint the data as they are querying it.

Question Two - Same Idea, Different Languages

Purpose:

In this question, you will explore how the same basic programming idea is expressed in different programming languages. The goal is not to master new languages, but to observe, compare, and explain how languages differ in syntax and design.

Learning Goals:

By completing this assignment , you will: - See how different languages express the same idea - Recognize similarities and differences in language design - Practice explaining code in plain English - Reflect on which language designs feel clearer or easier

What You Will Do:

1. Choose ONE Programming Concept From the List Below:

- **Variables**

- Loops
- Conditionals (if/else)
- Functions
- Data types
- Lists / arrays
- Errors (basic error handling)

2. Choose THREE Programming Languages

Select three different languages. You do **not** need to already know these languages well.

3. Generate Small Code Examples (Using AI Is Allowed)

You may use an AI tool to generate simple examples of your chosen concept in each language.

Guidelines:

- Each example should be basic and 5–10 lines of code maximum

Example prompt you might use: “Show a simple loop example in Ruby, Java, and Rust.”

You must include the code examples in your submission.

Prompt:

“Now I need to understand Variables in the context of 3 different programming languages than the 3 you provided initially. Show me code that highlights how variables work. Use the same example but written out in the 3 languages respectively.”

Response:

“Let’s build a clean, apples-to-apples comparison of **how variables work** in three *different* programming languages than before. To keep things consistent, we’ll use the **same example** in each language:

Store a number, store a piece of text, and then print a combined message.

I’ll use **C**, **Java**, and **Ruby** — three languages with very different philosophies around variables, which makes the comparison meaningful.

▣ 1. C — Variables Are Low-Level, Typed, and Memory-Bound

C requires you to **declare the type** of every variable explicitly.

You’re telling the computer exactly what kind of data to store and how much memory to allocate.

✦ Example in C

```
#include <stdio.h>

int main() { int age = 30; // integer variable
char name[] = "Bradley"; // character array (string)
printf("%s is %d years old.\n", name, age);
return 0; }
```

🧠 Key idea

C variables are like labeled boxes with a fixed size.

You must tell the compiler the box size before you use it.

▣ 2. Java — Variables Are Strongly Typed but Object-Oriented

Java also requires explicit types, but it treats many values as **objects**, not raw memory.

Variables belong to classes and methods, and Java enforces strict type rules.

✦ Example in Java

```
public class Main {
    public static void main(String[] args) {
        int age = 30; // integer variable
        String name = "Bradley"; // String object
        System.out.println(name + " is " + age + " years old.");
    }
}
```

🧠 Key idea

Java variables are more structured than C.

You still declare types, but Java manages memory for you and treats data as objects.

3. Ruby — Variables Are Dynamic and Flexible

Ruby does **not** require type declarations.

A variable can hold anything, and its type can change at runtime.

✦ Example in Ruby

```
age = 30 # dynamic variable
name = "Bradley" # also dynamic
puts "#{name} is #{age} years old."
```

🧠 Key idea

Ruby variables are just labels pointing to objects.

You don't declare types — Ruby figures it out as the program runs.

🔍 Side-by-Side Comparison

Lan gua ge	Variable Style	Must Declare Type?	Memory Handling	Example Feel
C	Low-level, fixed type	Yes	Manual/explicit	"Tell me exactly what this is and how big it is."
Java	Strongly typed, object-oriented	Yes	Automatic (garbage-collecte d)	"Tell me the type; I'll manage the rest."
Rub y	Dynamic, flexible	No	Automatic	"Just assign it; I'll figure it out."

If you want, I can expand this into:

- a deeper conceptual model of variable *scope*
- how constants differ from variables

- how each language handles memory under the hood
- or swap in different languages like Go, Rust, Kotlin, or PHP

Just tell me which direction you want to explore next.

5. Explain What You See (This Is the Most Important Part)

For each language, answer the following questions in your own words:

1. What looks similar for the three generated code examples to the other languages?
2. What looks different?
3. Which part is easiest to understand?
4. What is confusing or surprising?

There are no single “correct” answers. Clear explanations matter more than technical precision.

1. In all three languages, while they look very different, have a very similar logical flow.
 - a. Declare and age variable
 - b. Declare a name variable
 - c. Concatenate variables and strings into a sentence

2. C:

In C, at the end of the function it says `Return 0;` and although this looks similar to Java it is a very odd concept that the function to print the concatenated sentence has to return an INT of 0.

Java:

Java sort of looks a lot like C but it is interesting that it's function has to run inside of a Main function. While C is just calling and executing the function, Java would not execute the function standalone, it must run in the main function.

Ruby:

Ruby doesn't look anything like the other two. It looks more dynamically typed like Python. It's language looks more like spoken language rather than explicit syntax. The variables are dynamic and don't have to be explicitly declared.

3. C:

The easiest part to understand is what type the variables are themselves. They are explicitly called out at the variable creation so there is no question as to what the variable is or can be

Java:

Java is understandable in the same exact way C is. Each variable is immediately identifiable as to what it is as it's contained in the declaration of the variable itself, not relative to the context around it.

Ruby:

Ruby looks very readable and easy to understand. While the other 2 languages are 6 lines long or so, Ruby attains the same output with half the amount of lines.

4. C:

The concept of having to return a variable of declared size is confusing. The other 2 languages can just execute operations in the function without having to worry about the size of the action returned.

Java:

Java looks very technical. It's read across is not the easiest to follow. Although I can see how it's explicit layout and declarations can be helpful for projects with multiple people interrogating the same code or trying to follow the same practices, it seems like the learning curve would be very high making it not very accessible.

Ruby:

I am surprised how similar Ruby looks to Python. It is very intentional and can read across like regular language making it more accessible than the other 2 languages. The "put" command is odd though. Most languages have some sort of Print function, C is printf, java system.out.println, but Ruby says "puts". I would imagine this word is related conceptually to the syntax of the code, but at the surface level I couldn't readily identify what "puts" does.

Question Three - Explain This Program

Purpose:

This question focuses on understanding programming language concepts by carefully explaining how a real program works. You will use an LLM as a helper, but the goal is to demonstrate your own understanding of the language and its design ideas.

This is not a programming-heavy assignment. It is about reasoning, explanation, and clarity.

What You Will Do:

1. Choose a Program (≤ 50 lines)

Select a short program written in a language not used heavily in class. The program should involve at least one non-trivial concept (e.g., recursion, functional style, concurrency, logic rules)

To select a program, I went to Twitch and found a stream of a developer working in something that looked foreign to me. I found someone writing a compiler for Linux Systems and he was working in BASH. Here is a piece of a program I was able to find in his GitHub repo.

```
while [ $# -gt 0 ]; do

    case "$1" in

        -cfg)          config=$2; shift ;;

        -job)          build_only_this_job=$2; shift ;;

        -nodownload)  autodownload=0 ;;

        -noclearsrc)   noclearsrc="-noclearsrc" ;;

        -xtrace)       xtrace="-xtrace" ;;

        -v)           verbose="-v" ;;

        [0-9]*) build_only_this_job=$1 ;;

        *)            echo "Usage: $0 [ -nodownload ] [ -noclearsrc ] [ -cfg config ] \" \
```

```

        "[ -job <stage>-<package> ]"; exit 1 ;;

    esac

    shift

done

. scripts/parse-config

. scripts/functions.in

build_root="$base/build/$SDECFG_ID"

build_toolchain="$base/build/$SDECFG_ID/TOOLCHAIN"

build_logs="$build_toolchain/logs"; mkdir -p "${build_logs}"

build_pkgs="$build_toolchain/pkgs"; mkdir -p "${build_root}"

if [ "$SDECFG_PARANOIA_CHECK" = 1 ]; then

    scripts/Check-System || exit 1

fi

# Package Build loop - executed by build-target

#

pkgloop() {

    local x= y=

```

```

if [ "$SDECFG_NOBROKENDEPS" = 1 ]; then

    nobrokendeps="-nobrokendeps"

else

    nobrokendeps=""

fi


# crude pkg count

pkgs=$(sed -n '/^X 0/p;/^X .1/p;/^X ..2/p;/^X ...3/p;/^X ....4/p;/X .....5/p;

/^X .....6/p;/^X .....7/p;/^X .....8/p;/^X .....9/p' config/$config/packages | wc -l)


if [ -z "$build_only_this_job" ]; then

    if [ "`ls ${build_root}/var/adm/logs/*.err 2> /dev/null`" ]; then

        echo_header "Removing old error logs ..."

        for y in 0 1 2 3 4 5 6 7 8 9; do

            if [ "$SDECFG_RETRY_BROKEN" -eq 1 -o $y -le
$SDECFG_CONTINUE_ON_ERROR_AFTER ]; then

                for x in ${build_root}/var/adm/logs/$y-*.err; do

                    if [ -f $x ]; then

                        echo_status "Removing ${x#build_root} ..."

                        rm -f $x

```

```

                                fi
                        done
                fi
        done

fi

if [ "`ls ${build_root}/var/adm/logs/*.out 2> /dev/null`" ]; then

        echo_header "Removing old output logs ..."

        for x in ${build_root}/var/adm/logs/*.out; do

                echo_status "Removing ${x#${build_root}} ..."

                rm -f $x

        done

fi


# For all build stages, build all selected packages

local maxstage=5 stagelevel

[ "$SDECFG_DO_REBUILD_STAGE" = 1 ] && maxstage=9

[ "$SDECFG_CROSSBUILD" = 1 ] && maxstage=2

for stagelevel in $(seq 0 $maxstage); do

        pkgloop_stage $stagelevel

done

else

```

```

rm -f ${build_root}/var/adm/logs/${build_only_this_job}.{log,err}

local stagelevel=${build_only_this_job%%-*}

local STAGES='.....'

local match="^X  ${STAGES:0:$stagelevel}$stagelevel[^ ]*  [^ ]*  [^ ]*
${build_only_this_job#*-} "

next="$(grep "$match" < config/$config/packages)"

if [ ! "$next" ]; then

    echo_error "No job matching job spec: $build_only_this_job"

    exit 1

fi

pkgloop_package $stepi $stagelevel ${next#X }

exit 0

fi

local pkglst=`mktemp` errors=0; rm -f src/invalid-files.lst

echo_header "Searching for old lingering files ..."

sed '/^[^X]/d; s,.*=,,' config/$config/packages | cut -d' ' -f5 |

if [ $SDECFG_PKGFILE_VER = 1 ]; then

    while read p; do

        v=$(grep '^Package Name and Version:' \

```

```

        build/$SDECFG_ID/var/adm/packages/$p \
        2>/dev/null | cut -f6 -d' ')

        echo "$p-$v"

    done

else

    cat

fi > $pkglst

for file in $(ls build/$SDECFG_ID/TOOLCHAIN/pkgs/ 2> /dev/null); do

    x="$file"

    case $SDECFG_PKGFILE_TYPE in

        tar.*) x=${x%.$SDECFG_PKGFILE_TYPE} ;;

        none)      : ;;

    esac

    if ! grep -qx "$x" $pkglst && ! test "$x" = packages.db; then

        file="build/$SDECFG_ID/TOOLCHAIN/pkgs/$file"

        echo_error "$file should not be present" \

            "(now in src/invalid-files.lst)!"

        mkdir -p src; echo "$file" >> src/invalid-files.lst

        errors=1

    fi

done

```

```

        for
                                dir
                                in
build/$SDECFG_ID/var/adm/{cache,dependencies,descs,flists,md5sums,packages}; do

    for file in $(ls $dir 2> /dev/null); do

        if [ $SDECFG_PKGFILE_VER = 1 ]; then

            x="$file-"

        else

            x="$file"

        fi

        if ! grep -q "$x" $pkglst; then

            echo_error "$dir/$file should not be present (now in src/invalid-
files.lst)!"

            mkdir -p src; echo "$dir/$file" >> src/invalid-files.lst

            errors=1

        fi

    done

done

for file in $(ls build/$SDECFG_ID/var/adm/logs/); do

    x=""`echo $file | sed -e 's/^.-//' -e 's/\.log//' -e 's/\.err//' -e 's/\.out//`"

    if [ $SDECFG_PKGFILE_VER = 1 ]; then

        x=$x-

    else

        x=$x

    fi

```

```

        fi

        if ! grep -q "$x" $pkglst; then

            file="build/$SDECFG_ID/var/adm/logs/$file"

            echo_error "$file should not be present (now in src/invalid-files.lst)!"

            mkdir -p src; echo "$file" >> src/invalid-files.lst

            errors=1

        fi

    done

    [ $errors = 0 ] && echo_status "None found."

    rm $pkglst
}

# Parallel build handling

#

#

JOBS="$SDECFG_PARALLEL"

# Build command per component (could also be a single function with a case)

build() {

    local pkg=$1

```

```

local STAGES='.....'

local match="^X ${STAGES:0:$stagelevel}$stagelevel[^ ]* [^ ]* [^ ]* $pkg "

next="$(grep "$match" < config/$config/packages)"

if [ ! "$next" ]; then

    echo_error "No job matching job spec: $build_only_this_job"

    exit 1

fi

pkgloop_package $stepi $stagelevel ${next#X }

}

```

2. Use an LLM (Recommended)

Use an LLM to help you understand the program selected in the previous step. For example, you may ask: - What does this program do? - How does it work step by step? - What language features does it rely on?

Prompt:

“copy and paste code” + what language is this.

Response:

This is **Bash** — specifically a **Bourne-style shell script** used on Unix/Linux systems.

Everything in the snippet matches classic Bash scripting patterns:

🔗 Why this is Bash

1. Argument parsing with while [\$# -gt 0] and case

This is a textbook Bash idiom:

```
while [ $# -gt 0 ]; do case "$1" in -cfg) config=$2; shift ;;
```

2. Sourcing other scripts with .

```
. scripts/parse-config . scripts/functions.in
```

The . (dot) command is Bash shorthand for source.

3. Variable assignment without types

```
build_root="$base/build/$SDECFG_ID"
```

Bash variables are untyped strings unless used in arithmetic.

4. Use of backticks and command substitution

```
pkgs=$(sed -n ...)
```

5. Use of [test command

```
if [ "$SDECFG_PARANOIA_CHECK" = 1 ]; then
```

6. Functions defined with name() { ... }

```
pkgloop() { local x= y=
```

7. Heavy use of Unix utilities

sed, grep, wc, ls, rm, mkdir, etc.

8. Pattern matching inside case

```
[0-9]*) build_only_this_job=$1 ;;
```

9. Backtick-style command substitution

```
next="$(grep "$match" < config/$config/packages)"
```

🤖 What this script is doing (high-level)

This is a **build orchestration script** for a toolchain or package system. It:

- Parses command-line flags
- Loads configuration files
- Sets up build directories
- Iterates through build stages
- Removes old logs
- Validates package lists
- Calls functions like `pkgloop_stage` and `pkgloop_package`

It resembles scripts used in:

- Linux From Scratch build systems
- Embedded toolchain builders
- Custom distro build pipelines
- Legacy enterprise build systems

You must identify at least one explanation from the LLM that is: - Unclear, incomplete, or misleading, or - Correct but shallow or oversimplified

I wouldn't know if this was correct or not but this seems rather shallow. While it is giving code snippets of every other part of the program it just has a short one liner for this portion.

7. Heavy use of Unix utilities

`sed`, `grep`, `wc`, `ls`, `rm`, `mkdir`, etc.

3. Explain the Program Clearly

In your own words, explain: - What the program does - How it works - Which programming language concepts it uses. Your explanation should be clearer and more accurate than the LLM's.

Honestly, I cannot explain it any better than the LLM and I think that is what makes it so powerful. Concepts are so much more readily available and attainable. Before this very question I had no idea that there were such things as Linux Compilers for enterprise build systems, but now I could go through and learn the concepts step by step. To create an explanation clearer and more accurate than the LLM is impossible in my opinion. The more I learn about the topic the more it understands what I have learned and at any time would be able to deliver a response in the context and academia of my own knowledge better than I can.

With that being said, I use LLM's mainly as the most powerful tutor and trainer ever. After interrogating the code line by line and with the assistance of LLM I can actually speak to a concept that before 10 minutes ago I never knew existed.

Best I can tell, starting from 0 and only working with Copilot:

This particular portion of the script is a High level portion of a compiler that calls out and executes functions in proper order to achieve a desired result. Basically it is taking given arguments by the user and enacting functions upon those arguments in the appropriate order to achieve a build.
